

Reviewer Pack — Block-Composition Sub-Additivity of Coarse Depth κ via the R...

Entry 8047f7f8eea1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 01:39:16 UTC

Block-Composition Sub-Additivity of Coarse Depth κ via the Roe-Trace Pairing

Entry ID: 8047f7f8eea1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 01:39:16 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW); Roe-style coarse cohomology / coarse geometry of metric spaces (geometric group theory + index theory branch of analysis) **Field B** (complexity object): Boolean formula depth lower bounds for KRW-style block compositions $f \circ g$, the central complexity-theoretic object behind $P \not\subseteq NC^1$; equivalently the depth complexity $D(f \circ g)$ of the composed function in the Karchmer-Raz-Wigderson program

Statement:

For all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$ and $g: \{0,1\}^m \rightarrow \{0,1\}$ for which $HX^1(X_f)$ and $HX^1(X_g)$ admit nontrivial classes detected by Roe-controlled operators, the coarse depth κ defined via the trace_pairing on the KW-metric spaces obeys the sub-additive bound $\kappa(f \circ g) \geq \kappa(f) + \kappa(g) - C$, where C is a universal constant independent of n, m , and where $(X_{\{f \circ g\}}, d_{\{f \circ g\}})$ is built by the coarse_product operation. In particular, witness cocycles $c_f \boxtimes c_g \in HX^1(X_{\{f \circ g\}})$ and tensor Roe operators $T_f \otimes T_g \in C_u^*[X_{\{f \circ g\}}]$ realize $\log| \langle c_f \boxtimes c_g, T_f \otimes T_g \rangle | / \log(\text{propagation}(T_f \otimes T_g)) \geq \kappa(f) + \kappa(g) - C$.

Rationale (proposer's reasoning):

This sub-conjecture lifts axiom A2 (Composition sub-additivity for asdim) from the asymptotic-dimension layer to the strictly stronger κ layer, and chains through A1 ($\kappa \leq \text{depth}$) to recover the KRW conjecture $D(f \circ g) \geq D(f) + D(g) - O(1)$. The mechanism is the multiplicativity of the Roe-index trace pairing on tensor products: $\langle c_f \boxtimes c_g, T_f \otimes T_g \rangle = \langle c_f, T_f \rangle \cdot \langle c_g, T_g \rangle$ (Connes-style product), while the propagation of $T_f \otimes T_g$ equals $\text{propagation}(T_f) \cdot \text{propagation}(T_g)$ under the depth-additive metric of coarse_product. Taking logs yields the additive bound. Should this hold even on small toy compositions, it provides direct empirical support for the KRW direct-sum theorem at the κ level. It does not contradict A3 (anti-natural-proofs) because random f has $\kappa = O(1)$, so additivity holds trivially. It is consistent with A4 (anti-relativization) since coarse_product does not introduce oracle gates. A5 (Roe-index rigidity) underpins the stability of the tensor pairing under coarse equivalence.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2fe09bf387006c65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 25 ordered pairs $(f,g) \in \{\text{AND_2, OR_2, XOR_2, MAJ_3, IP_2}\}^2$, run 5 random seeds per pair for the Gaussian Roe kernel. For each pair compute $\text{slack}_{\{f,g\}} = \kappa(f) + \kappa(g) - \kappa(f \circ g)$ using the seed-median κ . The conjecture is SUPPORTED iff $\text{slack}_{\{f,g\}} \leq 2$ on $\geq 80\%$ of pairs ($\geq 20/25$) AND mean slack ≤ 2 across pairs. Control: median $\kappa(f \circ \text{rand}) - \kappa(f) \in [-1, 1]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.78	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.80	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson coarse cohomology Roe algebra formula depth - KRW composition conjecture coarse geometry trace pairing depth lower bound - uniform Roe algebra propagation boolean formula depth NC1

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def AND_2(x, y):
        return x and y

    def OR_2(x, y):
        return x or y

    def XOR_2(x, y):
        return x != y
```

```

def MAJ_3(x, y, z):
    return (x + y + z) >= 2

def IP_2(x, y):
    return x * y

functions = [AND_2, OR_2, XOR_2, MAJ_3, IP_2]

def hamming_distance(x, y):
    return sum(1 for i in range(len(x)) if x[i] != y[i])

def build_X_d(f, k):
    n = 2 ** k
    X = [tuple(i) for i in range(n)]
    d = [[hamming_distance(x, y) for y in X] for x in X]
    return X, d

def build_basis(X, d):
    n = len(X)
    C0 = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    C1 = []
    for x in X:
        row = [0] * n
        for y in X:
            if hamming_distance(x, y) == 1:
                row[y] = 1
        C1.append(row)
    delta = subtract_matrices(C1, multiply_matrices(delta_image(C0), transpose_matrix(C1)))
    cokernel = find_cokernel(delta)
    basis = [cokernel[i][0] for i in range(len(cokernel)) if any(cokernel[i])]
    return basis

def delta_image(C0):
    n = len(C0)
    image = []
    for x in range(n):
        row = [sum(C0[x][j] * C1[j][y] for j in range(n)) for y in range(n)]
        image.append(row)
    return image

def transpose_matrix(M):
    return [[M[j][i] for j in range(len(M))] for i in range(len(M[0]))]

def subtract_matrices(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def multiply_matrices(A, B):
    n = len(A)
    m = len(B[0])
    result = [[sum(A[i][k] * B[k][j] for k in range(m)) for j in range(m)] for i in range(n)]
    return result

def find_cokernel(M):
    n = len(M)
    rank = 0
    cokernel = []

```

```

    for i in range(n):
        if any(M[i]):
            row = [M[i][j] for j in range(n) if M[i][j]]
            cokernel.append(row)
            rank += 1
    return cokernel

def build_Roe_operator(X, d, R):
    n = len(X)
    T = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if hamming_distance(X[i], X[j]) <= R:
                T[i][j] = random.gauss(0, 1)
    return symmetrize_matrix(T)

def symmetrize_matrix(M):
    n = len(M)
    for i in range(n):
        for j in range(i + 1, n):
            M[i][j] += M[j][i]
            M[j][i] = M[i][j]
    return M

def trace(T):
    n = len(T)
    return sum(T[i][i] for i in range(n))

def propagation(T):
    n = len(T)
    max_propagation = 0
    for i in range(n):
        for j in range(i + 1, n):
            if T[i][j] != 0:
                max_propagation = max(max_propagation, hamming_distance(X[i], X[j]))
    return max_propagation

def coarse_product(X_f, d_f, X_g, d_g):
    n = len(X_f)
    m = len(X_g)
    X_fg = [(x, y) for x in X_f for y in X_g]
    d_fg = [[max(d_f[x[0]][y[0]], d_g[x[1]][y[1]]) for y in X_fg] for x in X_fg]
    return X_fg, d_fg

def trace_pairing(c, T):
    n = len(c)
    result = 0
    for i in range(n):
        for j in range(i + 1, n):
            if c[i][j] != 0:
                result += c[i][j] * T[i][j]
    return result

def log_ratio(trace_value, propagation_value):
    return math.log(abs(trace_value)) / math.log(propagation_value)

```

```

def run_composition(f, g, k_f, k_g):
    X_f, d_f = build_X_d(f, k_f)
    basis_f = build_basis(X_f, d_f)
    T_f = build_Roe_operator(X_f, d_f, 4)
    kappa_f = max(log_ratio(trace_pairing(basis_f[i], T_f), propagation(T_f)) for i in range(10))

    X_g, d_g = build_X_d(g, k_g)
    basis_g = build_basis(X_g, d_g)
    T_g = build_Roe_operator(X_g, d_g, 4)
    kappa_g = max(log_ratio(trace_pairing(basis_g[i], T_g), propagation(T_g)) for i in range(10))

    X_fg, d_fg = coarse_product(X_f, d_f, X_g, d_g)
    basis_fg = build_basis(X_fg, d_fg)
    T_fg = build_Roe_operator(X_fg, d_fg, 4)
    kappa_fg = max(log_ratio(trace_pairing(basis_fg[i], T_fg), propagation(T_fg)) for i in range(10))

    return kappa_f, kappa_g, kappa_fg

def run_control(f, k):
    X_f, d_f = build_X_d(f, k)
    basis_f = build_basis(X_f, d_f)
    T_f = build_Roe_operator(X_f, d_f, 4)
    kappa_f = max(log_ratio(trace_pairing(basis_f[i], T_f), propagation(T_f)) for i in range(10))

    X_rand = [tuple(random.randint(0, 1) for _ in range(k)) for _ in range(2 ** k)]
    d_rand = [[hamming_distance(x, y) for y in X_rand] for x in X_rand]
    basis_rand = build_basis(X_rand, d_rand)
    T_rand = build_Roe_operator(X_rand, d_rand, 4)
    kappa_rand = max(log_ratio(trace_pairing(basis_rand[i], T_rand), propagation(T_rand)) for i in range(10))

    return kappa_f, kappa_rand

def check_additivity(kappa_f, kappa_g, kappa_fg):
    C = 2
    return kappa_fg >= kappa_f + kappa_g - C

def check_control(kappa_f, kappa_rand):
    return abs(kappa_rand - kappa_f) <= 1

results = []
for f in functions:
    for g in functions:
        k_f = random.randint(1, 4)
        k_g = random.randint(1, 4)
        kappa_f, kappa_g, kappa_fg = run_composition(f, g, k_f, k_g)
        kappa_f_rand, kappa_rand = run_control(f, k_f)

        results.append({
            "f": f.__name__,
            "g": g.__name__,
            "k_f": k_f,
            "k_g": k_g,
            "kappa_f": kappa_f,
            "kappa_g": kappa_g,
            "kappa_fg": kappa_fg,
            "kappa_f_rand": kappa_f_rand,

```

```

        "kappa_rand": kappa_rand
    })

support_count = 0
total_slack = 0

for result in results:
    if check_additivity(result["kappa_f"], result["kappa_g"], result["kappa_fg"]):
        support_count += 1
        slack = result["kappa_f"] + result["kappa_g"] - result["kappa_fg"]
        total_slack += slack

mean_slack = total_slack / len(results)
support_fraction = support_count / len(results)

if support_fraction >= 0.8 and mean_slack <= 2:
    return {
        "metric_name": "slack",
        "metric_value": mean_slack,
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    for result in results:
        if not check_additivity(result["kappa_f"], result["kappa_g"], result["kappa_fg"]):
            return {
                "metric_name": "slack",
                "metric_value": mean_slack,
                "instances_tested": len(results),
                "conjecture_holds": False,
                "counterexample": f"({result['f']}, {result['g']}) with kappa_f={result['kappa_f']}, kappa_g={result['kappa_g']}, kappa_fg={result['kappa_fg']}"
            }

    return {
        "metric_name": "slack",
        "metric_value": mean_slack,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if not sys.argv[1:] else [int(s) for s in sys.argv[1:]]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")

    results = []
    for seed in seeds:
        with open(f"trial_{seed}.json", "r") as f:
            trial_results = json.load(f)
            results.extend(trial_results)

    support_count = 0

```

```

total_slack = 0

for result in results:
    if check_additivity(result["kappa_f"], result["kappa_g"], result["kappa_fg"]):
        support_count += 1
        slack = result["kappa_f"] + result["kappa_g"] - result["kappa_fg"]
        total_slack += slack

mean_slack = total_slack / len(results)
support_fraction = support_count / len(results)

if support_fraction >= 0.8 and mean_slack <= 2:
    print(f"RESULT: SUPPORTED mean={mean_slack} std=0.0 support_fraction={support_fraction}")
elif any(not check_additivity(result["kappa_f"], result["kappa_g"], result["kappa_fg"]) for result in results):
    first_failing_seed = next((seed for seed, result in enumerate(results) if not check_additivity(result["kappa_f"], result["kappa_g"], result["kappa_fg"])), None)
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6db712c6.py", line 244, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6db712c6.py", line 186, in run_trial
    kappa_f, kappa_g, kappa_fg = run_composition(f, g, k_f, k_g)
                                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6db712c6.py", line 143, in run_composition
    X_f, d_f = build_X_d(f, k_f)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6db712c6.py", line 43, in build_X_d
    X = [tuple(i) for i in range(n)]
          ~~~~~^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`'int' object is not iterable`) at `build_X_d` before any `k` values were computed, so no data exists to evaluate the pre-registered support criterion. | next: Fix `build_X_d` to enumerate vertices correctly (e.g., `X = [tuple(b) for b in itertools.product([0,1], repeat=n)]`) and rerun the 25-pair $\times$ 5-seed protocol.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	9372
2	novelty	claude_max	opus	0	0	3403
3	novelty	claude_max	opus	0	0	6682
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24093
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29271
6	judge	claude_max	opus	0	0	6066

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78887 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/8047f7f8eea1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8047f7f8eea1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8047f7f8eea1.tar.gz` (if generated)